

37 — Search Agent with Tool-Use Pattern

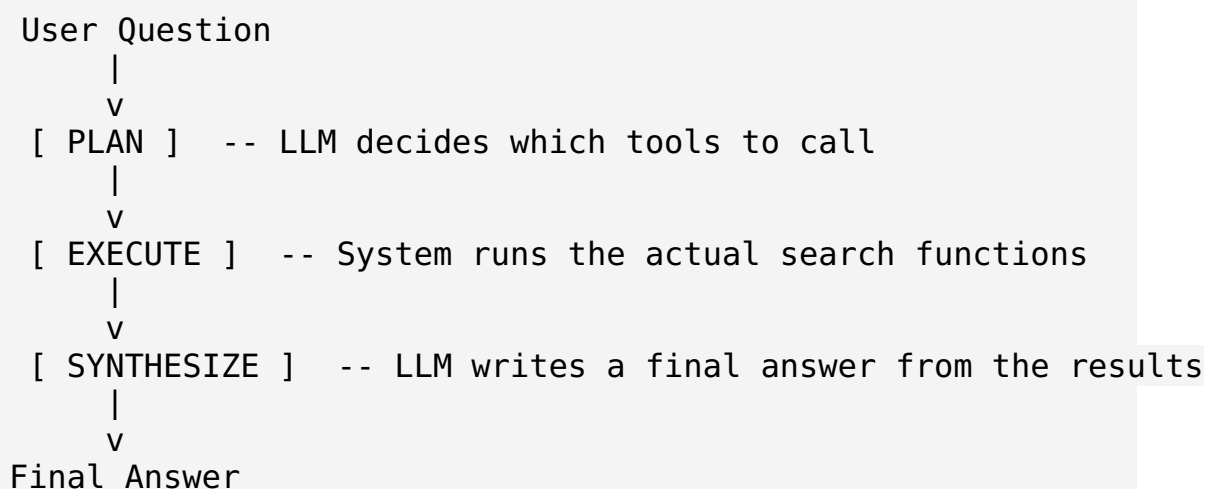
What It Is

A tool-use agent is an LLM that does not answer questions directly. Instead, it follows a three-step loop:

1. **Plan** — The LLM reads the user's question and decides which tools (functions) it needs to call. It outputs a structured plan (JSON) listing the tool names and arguments.
2. **Execute** — The system runs each tool the LLM requested, collecting real results from databases, APIs, or services.
3. **Synthesize** — The LLM receives the raw tool results and writes a final, human-readable answer that combines everything.

The LLM never touches the database itself. It acts as a coordinator: it understands the question, picks the right tools, and summarizes what comes back.

This is the same core pattern behind ChatGPT plugins, Claude's tool use, and most production AI assistants. The key insight is **separation of concerns** — the LLM handles natural language understanding and generation, while deterministic code handles data retrieval. The LLM is good at understanding intent; the tools are good at fetching accurate data.



Real-Life Analogy

Think of a hotel concierge. You walk up and ask: "Where should I eat tonight?"

The concierge does not guess. Instead, they:

1. **Check their restaurant guide** (tool 1) — which restaurants are nearby, what cuisine do they serve?
2. **Call the restaurant** (tool 2) — is there a table available at 8pm?
3. **Give you a recommendation** (synthesis) — "I'd suggest Trattoria Roma, they have a table at 8pm, and their pasta is excellent for your budget."

The concierge is the coordinator, not the database. They know how to ask the right questions and combine the answers into something useful. They do not memorize every restaurant's menu — they know where to look it up.

Our search agent works the same way. It does not contain search results. It knows which search endpoints exist, understands what each one is good at, and routes your question to the right one(s).

Why We Used It Here

The project has three different search endpoints, each designed for a different type of query:

Endpoint	Purpose
<code>search_categories</code>	Find product/service categories (e.g., "What pump categories exist?")
<code>search_companies</code>	Find matching companies (e.g., "Find companies that make pumps")
<code>search_single_company</code>	Search within one company's data (e.g., "What does company #42 sell?")

Without the agent, the user (or frontend developer) must know which endpoint to call for each question. That is a poor user experience — you have to understand the API to use it.

With the agent, you just ask a natural language question:

- "Find companies that make industrial pumps" --> agent calls `search_companies`
- "What welding categories exist?" --> agent calls `search_categories`
- "Which companies offer CNC machining, and what categories does that fall under?" --> agent calls **both** `search_companies` and `search_categories`

The agent handles the routing decision, so the user never needs to know about the underlying API structure.

Code Walkthrough

1. The Tool Catalog — `TOOLS_DESCRIPTION`

File: `search_api/services/search_agent.py` , lines 22-38

```
TOOLS_DESCRIPTION = """You have access to these search tools:

1. search categories(query, language, limit) – Search product/service
   Use when: the user asks about types of products, services, or categories.

2. search companies(query, language, limit) – Find matching companies.
   Use when: the user asks about companies, suppliers, or manufacturers.

3. search single company(company id, query, language) – Search within a company.
   Use when: the user asks about a specific company by ID.

Rules:
- You may call multiple tools if the question spans both categories and companies.
- Set language to "hu" for Hungarian queries, "en" for English, or null for any language.
- Return your plan as JSON with this schema:
  {"tools": [{"name": "...", "args": {"query": "...", ...}}]}
"""
```

This string is the "menu" given to the LLM. It tells the model what tools exist, when to use each one, and what format to return. The LLM never sees the actual Python functions — it only sees this description and must output a JSON plan that matches the schema.

Key design choice: the description includes **"when to use"** hints. Without these, the LLM would have to guess purely from the function name, which leads to worse routing accuracy.

2. Planning — `_plan_tools()`

File: `search_api/services/search_agent.py` , lines 48-69

```
def plan_tools(query: str) -> list[dict]:
    """Ask the LLM which tools to call for this query."""
    llm = get_llm_provider()

    plan_prompt = (
        f"{TOOLS_DESCRIPTION}\n\n"
        f"User question: {query}\n\n"
        f"Return ONLY a JSON object with your tool plan. No explanation."
    )
```

```

raw = generate_with_retry(llm, plan_prompt, system_prompt="You are
data, errors = validate_json_response(raw, required_fields=["tools
if errors or data is None:
    logger.warning(f"Agent plan validation failed: {errors}. Falli
    return [{"name": "search_categories", "args": {"query": query,

tools = data.get("tools", [])
if not tools or not isinstance(tools, list):
    return [{"name": "search_categories", "args": {"query": query,

return tools

```

Step by step:

1. **Get the LLM** — `get_llm_provider()` returns whichever provider is configured in the Admin UI (Groq, OpenRouter, or Gemini). The agent does not care which LLM is behind the scenes.
2. **Build the prompt** — Combines the tool catalog with the user's question and a strict instruction to return JSON only.
3. **Call the LLM** — `generate_with_retry` handles transient failures (rate limits, timeouts).
4. **Validate the response** — `validate_json_response` (from the guardrails module, see learning doc #34) parses the raw text and checks that it contains a `"tools"` field. If the LLM returns malformed JSON or misses the field, we catch it here.
5. **Fallback** — If anything goes wrong, instead of crashing, the agent falls back to a safe default: search categories with the original query. This ensures the user always gets some result.

3. Execution — `_execute_tool()`

File: `search_api/services/search_agent.py` , lines 72-115

```

def execute_tool(tool: dict) -> dict:
    """Execute a single tool call and return results."""
    name = tool.get("name", "")
    args = tool.get("args", {})

    if name == "search_categories":
        table = "category vectors"
    elif name == "search_companies":
        table = "company vectors"
    elif name == "search_single_company":
        table = "company_vectors"
    else:
        return {"tool": name, "error": f"Unknown tool: {name}", "result": None}

    try:

```

```

        raw = search_table(
            table=table,
            query=args.get("query", ""),
            limit=args.get("limit", 5),
            lang_override=args.get("language"),
            rerank=True,
        )
        # ... summarize results ...
    except Exception as e:
        return {"tool": name, "error": str(e), "results": []}

```

This is the deterministic part — no LLM involved. It takes the tool name from the plan, maps it to the correct database table, and calls the existing `search_table` function (which does the RRF hybrid search we built earlier).

Key design points:

- **Reuses existing search infrastructure** — `search_table` already handles vector search, BM25, RRF fusion, cross-language fallback, and reranking. The agent does not duplicate any of that.
- **Unknown tool handling** — If the LLM hallucinates a tool name that does not exist, the function returns an error dict instead of crashing.
- **Result truncation** — Each result's text is capped at 300 characters. This keeps the synthesis prompt within token limits and avoids wasting LLM context on redundant detail.

4. Synthesis — `_synthesize()`

File: `search_api/services/search_agent.py`, lines 118-143

```

def synthesize(query: str, tool results: list[dict]) -> str:
    """Ask the LLM to synthesize tool results into a final answer."""
    llm = get_llm_provider()

    context parts = []
    for tr in tool results:
        if tr.get("error"):
            context parts.append(f"Tool '{tr['tool']}' failed: {tr['error']}")
            continue
        context parts.append(f"Tool '{tr['tool']}' returned {tr['results']}")
        for r in tr["results"]:
            context parts.append(
                f"  - [{r['id']}] ({r['language']}, score={r['score']}: {r['text']}")

    context = "\n".join(context_parts)
    synthesis prompt = (
        f"Search results:\n\n{context}\n\n---\n"

```

```

        f"User question: {query}\n\nAnswer the question based on the s
    )
    return generate_with_retry(llm, synthesis_prompt, system_prompt=SY

```

The synthesis step formats all tool results into a readable context block, then asks the LLM to write a final answer. The system prompt (`SYNTHESIS_PROMPT`) instructs the LLM to:

- Be concise
- Cite specific IDs when relevant
- Be honest about empty results
- Match the language of the question (Hungarian question gets Hungarian answer)

5. Orchestration — `run_search_agent()`

File: `search_api/services/search_agent.py` , lines 146-192

```

def run_search_agent(query: str) -> dict:
    """Full agent loop: plan -> execute -> synthesize."""
    start = time.perf_counter()

    plan = plan_tools(query)          # Step 1: Plan
    plan_time = (time.perf_counter() - start) * 1000

    tool_results = []
    for tool in plan:
        result = execute_tool(tool)   # Step 2: Execute each tool
        tool_results.append(result)
    exec_time = (time.perf_counter() - start) * 1000 - plan_time

    answer = _synthesize(query, tool_results) # Step 3: Synthesize

    total_time = (time.perf_counter() - start) * 1000

    return {
        "answer": answer,
        "plan": plan,
        "tool_results": [...],
        "timing": {
            "plan ms": ...,
            "execution ms": ...,
            "synthesis ms": ...,
            "total_ms": ...,
        },
    }

```

This is the main loop that ties the three steps together. It also tracks timing for each phase — plan, execution, and synthesis — which is

returned in the response and displayed in the UI. The `EVIDENCE_*` log markers provide UAT proof that the agent ran correctly.

6. The API Endpoint

File: `search_api/routers/agent.py`

```
class AgentRequest(BaseModel):
    query: str = Field(..., min_length=1, description="Natural language query")

@router.post("/agent/ask")
def agent_ask(req: AgentRequest):
    return run_search_agent(req.query)
```

A minimal FastAPI endpoint. The `AgentRequest` model validates that the query is not empty. The endpoint returns the full agent response: answer, plan, tool results, and timing.

7. The Chat UI — `SearchAgent.jsx`

File: `admin-ui/src/pages/SearchAgent.jsx`

The frontend is a chat interface with these key features:

- **Message history** — Messages are stored in React state as an array of `{role, text, plan, toolResults, timing}` objects.
- **AgentBubble component** — Renders assistant messages with an expandable "Agent Reasoning" section:
- **Plan badges** — Shows which tools the agent chose (e.g., `search_companies(industrial pumps)`)
- **Execution results** — Shows per-tool result counts or errors, color-coded (green for success, red for error)
- **Timing breakdown** — Badges showing Plan/Search/Synthesis/Total milliseconds
- **Suggested queries** — Pre-filled example questions so users can try the agent immediately
- **Loading state** — Shows "Agent thinking... planning tools..." while waiting for the response

The expandable reasoning section is important for transparency — users can see exactly what the agent did, not just the final answer. This builds trust and helps debug unexpected results.

How to Explain This in an Interview / to a Teammate

"We built a search agent using the tool-use pattern. Instead of making the user pick which search endpoint to call, the agent takes a natural language question and asks an LLM to plan which tools are needed —

categories search, company search, or both. It then executes those searches using our existing RRF hybrid search infrastructure and asks the LLM to synthesize the results into a final answer. The whole thing has guardrails — if the LLM returns a malformed plan, we fall back to a default category search so the user always gets something. The frontend shows an expandable reasoning panel so users can see exactly which tools were called, how many results each returned, and how long each step took."